Design and Verification of a Dynamic Branch Prediction Unit

Shubhaang, Tanish, Tanmay, Vansh, and Lakshya
Phase I review draft. Full names, affiliations and section approvals pending.

Abstract—A synthesizable Verilog branch prediction unit combines two-bit direction prediction, a tagged target buffer, speculative next-PC selection and recovery. Directed tests exercise training, aliasing, invalid resolution and redirect priority. A separate direction experiment compares three predictors on 130 branches across three synthetic workloads. The two-bit predictor correctly predicts 55 branches (42.31%) and performs best on the loop trace. Generic Yosys synthesis checks seven RTL tops. The CPI model is illustrative and maximum operating frequency remains unmeasured.

Index Terms—branch prediction, branch target buffer, RTL verification, Yosys

I. Introduction

Conditional branches can change the fetch address before their outcome is available. Direction and target predictions let fetch select a speculative address. When the saved prediction disagrees with resolution, the next-PC path redirects and signals a flush. This project implements that interface in a Verilog subsystem with independently testable components [1].

II. Literature Review

Smith [2] compared static and history-based predictors using instruction traces. A one-bit last-outcome predictor changes its decision after one contrary outcome. Two-bit saturation adds hysteresis and can retain a taken prediction across an isolated loop exit. Accuracy depends on the outcome sequence and table indexing. Lee and Smith [3] studied design tradeoffs of retaining branch targets. In our prototype, selecting the stored target requires a taken direction prediction and a hit in the tagged target buffer.

III. Proposed Solution

The wrapper connects the existing direction, target, next-PC and recovery modules. The default 16-entry branch history table indexes aligned fetch-PC bits. Each two-bit counter resets to weakly not taken, increments on taken outcomes and decrements on not-taken outcomes, saturating at each endpoint. The high bit selects direction. Distinct PCs may alias in this untagged table.

The direct-mapped branch target buffer stores a valid bit, PC tag and target per entry. A different tag at the same index replaces the entry on a qualifying write. An invalid entry or tag mismatch produces a miss. A taken prediction with a hit selects the buffered target. Other lookups select PC+4.

TABLE I
Correct direction predictions by synthetic workload

Workload	Branches	Always NT	1-bit	2-bit
Loop	40	8	24	31
Alternating	40	20	0	0
Mixed BEQ/BNE	50	20	23	24
Overall	130	48	47	55
Accuracy (%)		36.92	36.15	42.31

At resolution, the caller supplies the original saved direction, hit and target, along with the actual outcome and target. A recovery redirect overrides live speculation. Valid resolved branches train direction. Valid taken branches train the target buffer. Reset inhibits training. Invalid or squashed resolutions cause neither training nor recovery. The caller retains prediction metadata and distributes flushes to pipeline registers. Figure 1 shows the data paths.

IV. Implementation and Verification

Icarus simulation exercises counter training, BTB replacement, next-PC and recovery behavior, and direction comparison. Figure 2 plots VCD values from a fresh make test run on ee7f45c. The wrapper tests 16-entry and 32-entry configurations, each with 70 passing checks. The suite covers wrong targets, saved metadata during unrelated fetch, invalid branches and recovery priority [1].

V. Results and Limitations

The HDL testbench and independent Python oracle agree on the prediction counts for all 130 branches. Table I and Fig. 3 use the committed counts. The overall row weights workloads by their branch counts. The two-bit predictor obtains $31 + 0 + 24 = 55$ correct out of 130, or 42.307692%. Always not taken wins the alternating workload. The three deterministic workloads are synthetic and start from reset. Results describe these selected traces.

The illustrative model uses $CPI = 1 + 0.2 \times (1 - \text{accuracy}) \times 3$, with accuracy as a fraction. It assumes ideal CPI 1, branch fraction 0.2 and three cycles per direction miss. It yields 1.378462, 1.383077 and 1.346154 for always not taken, one bit and two bits, respectively. It omits BTB target misses and other pipeline costs. The experiment supplies no measured processor CPI or speedup.

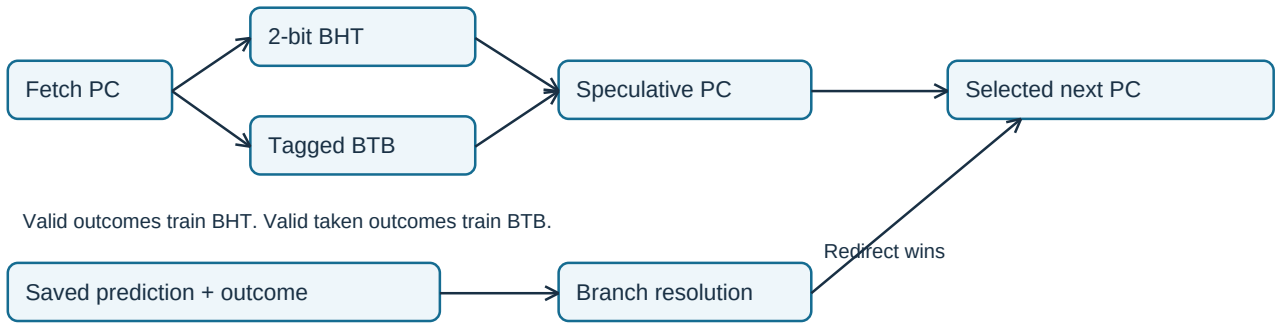

Fig. 1. Integrated BPU data paths derived from the merged wrapper at revision ee7f45c. The caller supplies saved prediction metadata and outcomes.

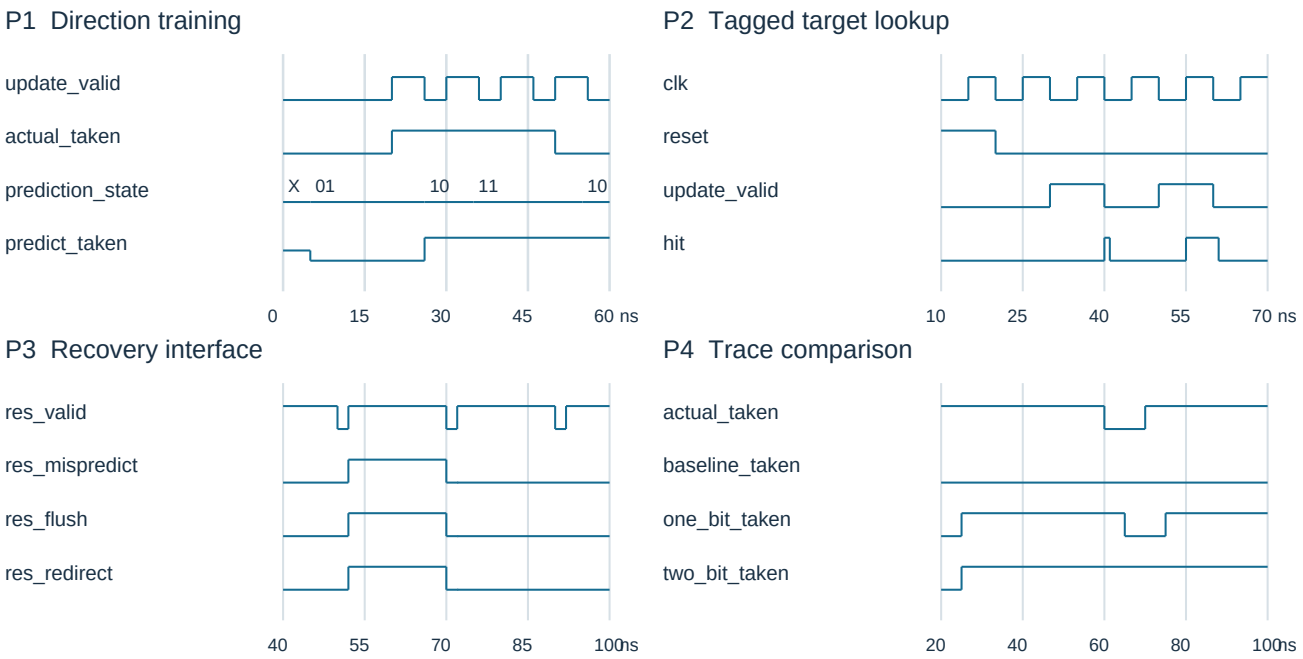

Fig. 2. Icarus VCD excerpts from P1 p1_bht.vcd, P2 btb.vcd, P3 p3_integrated.vcd and P4 p4_comparison.vcd. Times are simulation stimulus in ns. P1 labels show the binary counter state.

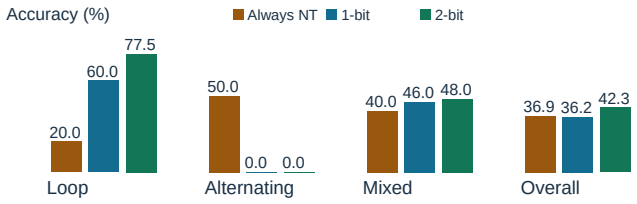

Fig. 3. Direction accuracy from committed accuracy.csv. Overall values are weighted by branch count.

TABLE II
Generic cells from Yosys 0.33, CI run 42200408851

RTL top	Cells
predictor_2bit	162
predictor_1bit	88
branch_target_buffer	2498
next_pc_mux	112
branch_resolution	214
predictor_top	227
branch_prediction_unit	2993

Table II transcribes the seven-top synthesis summary. Yosys 0.33 (2584903a060) ran hierarchy checks, flattened synthesis and check -assert. Counts are generic cells for independently synthesized tops. Device-mapped FPGA resources and post-route timing require a selected device and constraints. Fmax remains unmeasured.

VI. Individual Contributions

Shubhaang (P1) developed the saturating predictor and verification, and integrated the final wrapper. Tanish (P2) developed and verified the tagged target buffer. Tanmay (P3) implemented speculative selection and recovery, including the interface integration testbench. Vansh (P4) developed baseline comparison, trace verification and the independent oracle. Lakshya (P5) implemented reproducible synthesis and derived accuracy/CPI evidence, and assembled the draft. Attribution follows the P1/P2 design notes, P3–P5 report sections and merged PR history [1]. Each author will review their section before release.

References

[1] MCA Project 5 team, Dynamic Branch Prediction Unit, revision ee7f45c, Nov. 2026. Source and evidence. Synthesis CI. Integration CI.

[2] J. E. Smith, A Study of Branch Prediction Strategies, in Proc. 8th Int. Symp. Computer Architecture, 1981, pp. 135–148. Original paper.

[3] J. K. F. Lee and A. J. Smith, Analysis of Branch Prediction Strategies and Branch Target Buffer Design, Univ. California, Berkeley, Tech. Rep. UCB/CSD-83-121, Aug. 1983. Original report.